Docker Commands

Docker Commands

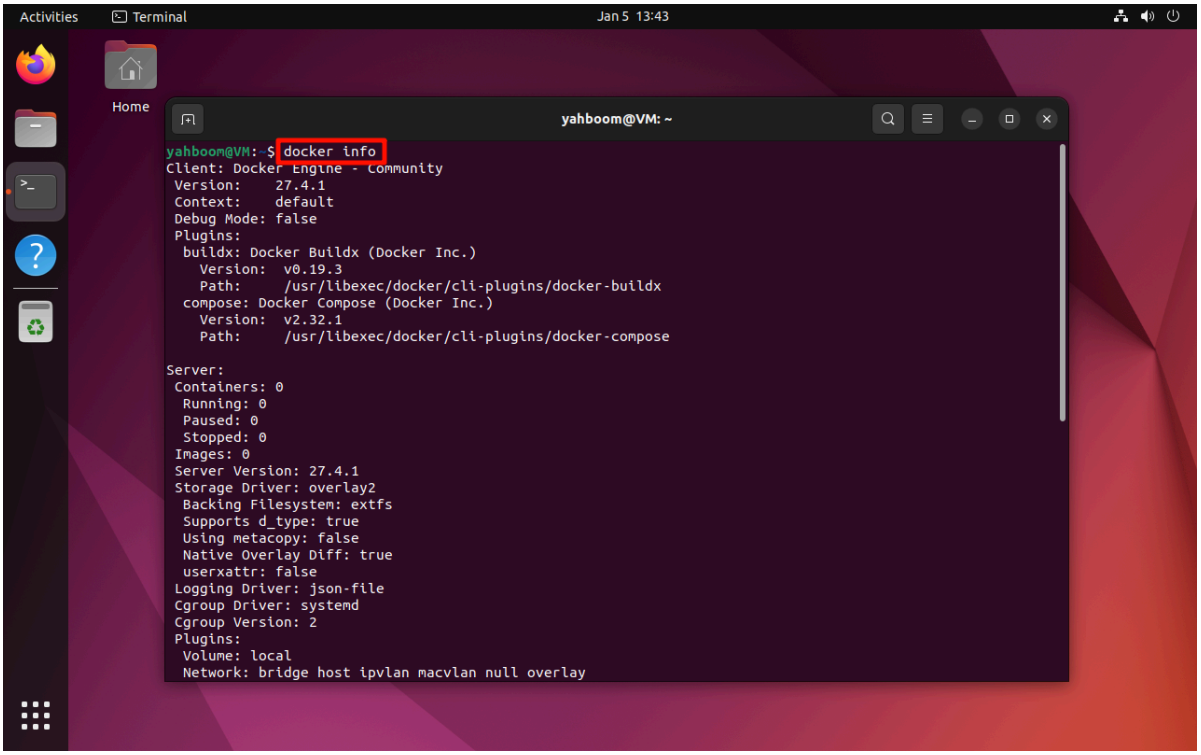
1. View Detailed Information
2. View the Version Number
3. Pull the Image
4. Run the Container
    4.1. View the Running Containers
    4.2. View All Containers (Including Stopped Containers)
5. Clean Up Stopped Containers
6. View Local Images
7. Delete Images
8. Commit the Image
9. Stop the Container
10. Multiple Terminals Entering the Same Container

The Docker engine includes Docker CLI, which provides command-line tools for interacting with the Docker daemon. This tutorial introduces the usage of common Docker commands.

1. View Detailed Information

> **docker info**

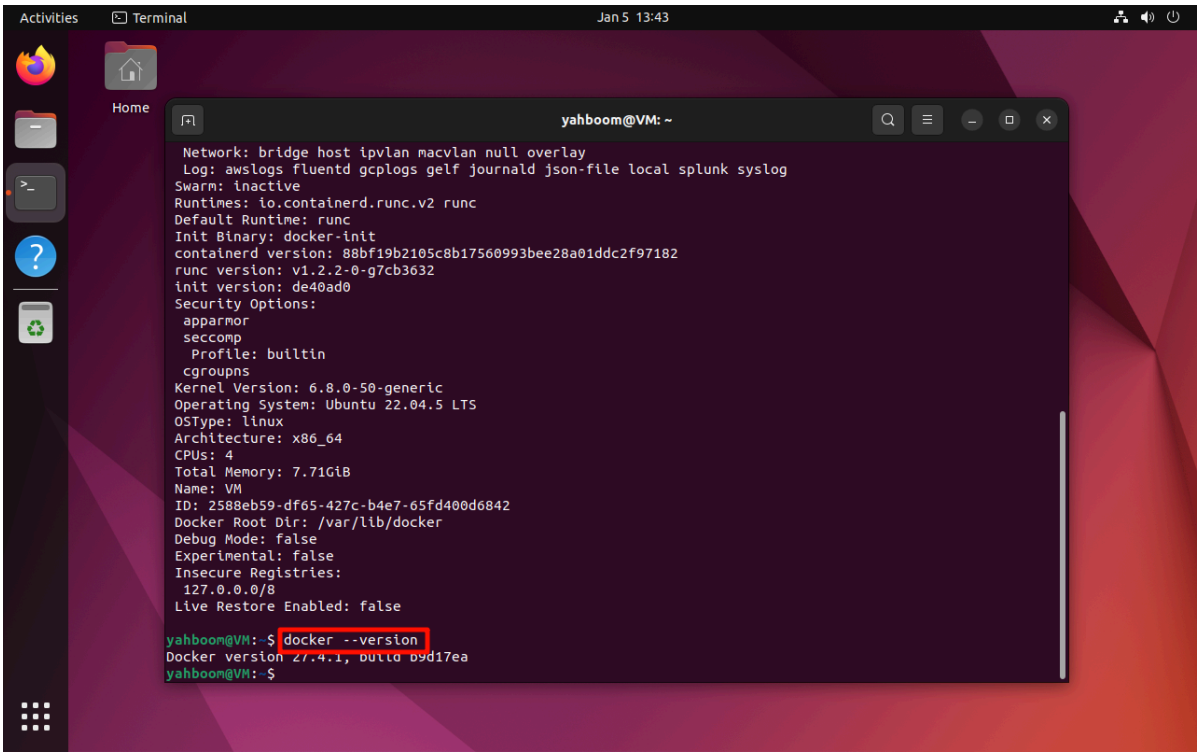
```
docker info
```



2. View the Version Number

> **docker --version**

```
docker --version
```



3. Pull the Image

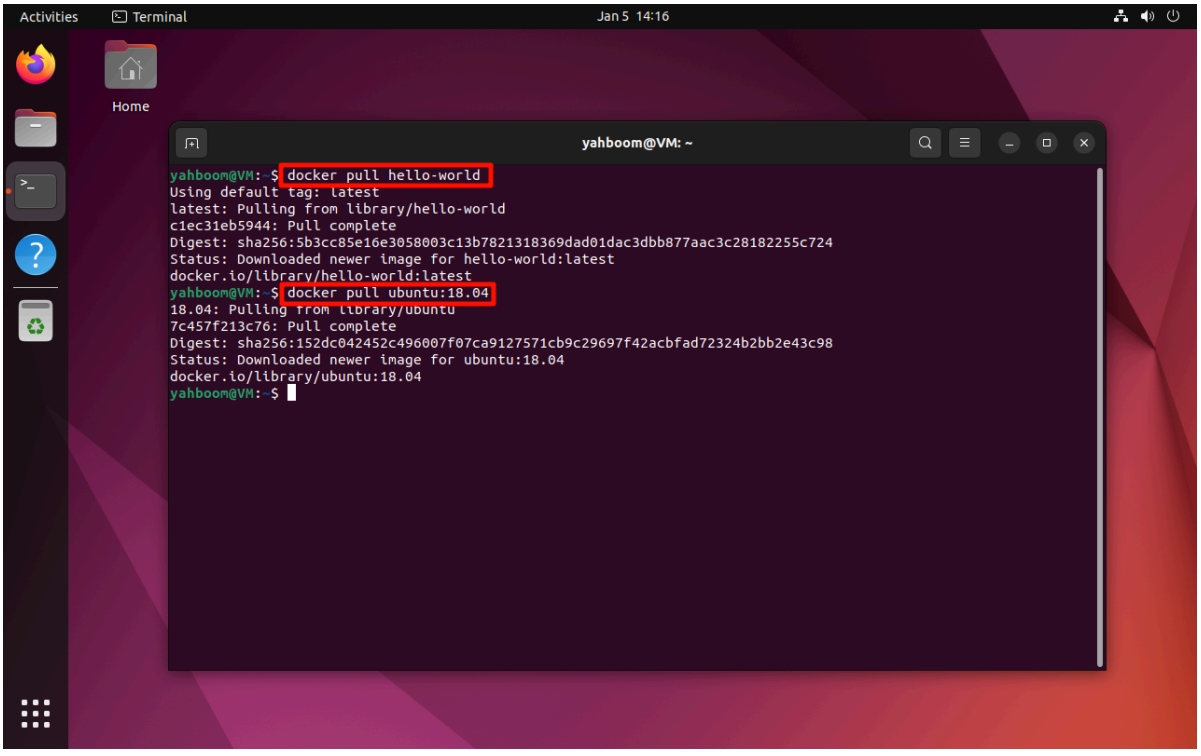
> **docker pull <image_name>**

If no tag is specified, the image with the `latest` tag is pulled by default.

```
docker pull hello-world
```

> **docker pull <image_name>:**

```
docker pull ubuntu:18.04
```



4. Run the Container

If the image to be run does not exist locally, docker pulls the corresponding image automatically.

> **docker run <image_name>**

```
docker run hello-world
```

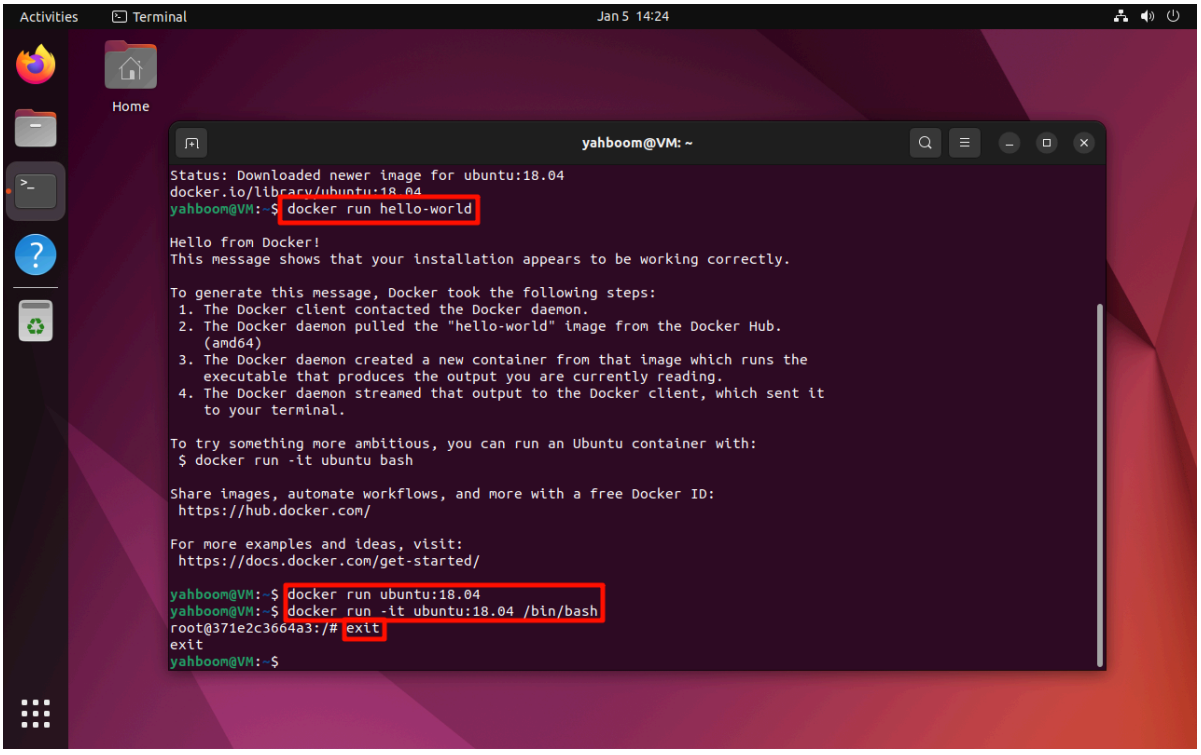
> **docker run <image_name>:**

Run the container once and let it exit:

```
docker run ubuntu:18.04
```

Run the container interactively; type `exit` to exit

```
docker run -it ubuntu:18.04 /bin/bash
```



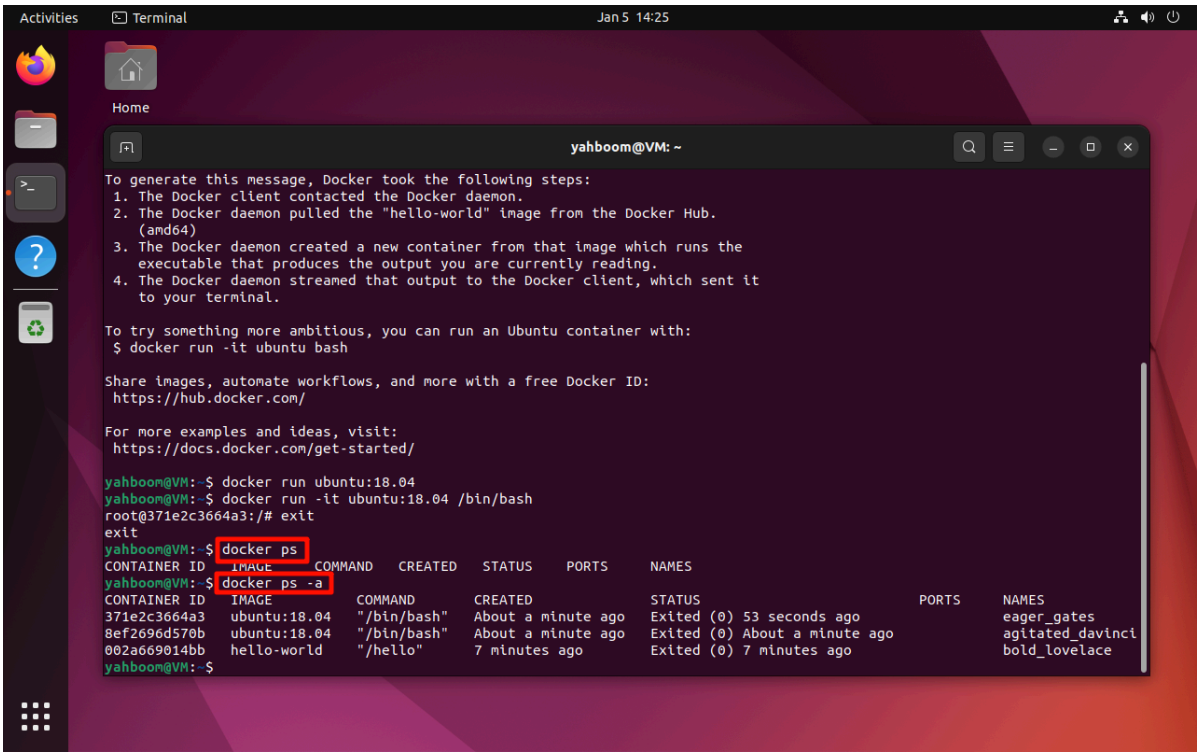
4.1. View the Running Containers

> **docker ps**

```
docker ps
```

4.2. View All Containers (Including Stopped Containers)

```
docker ps -a
```

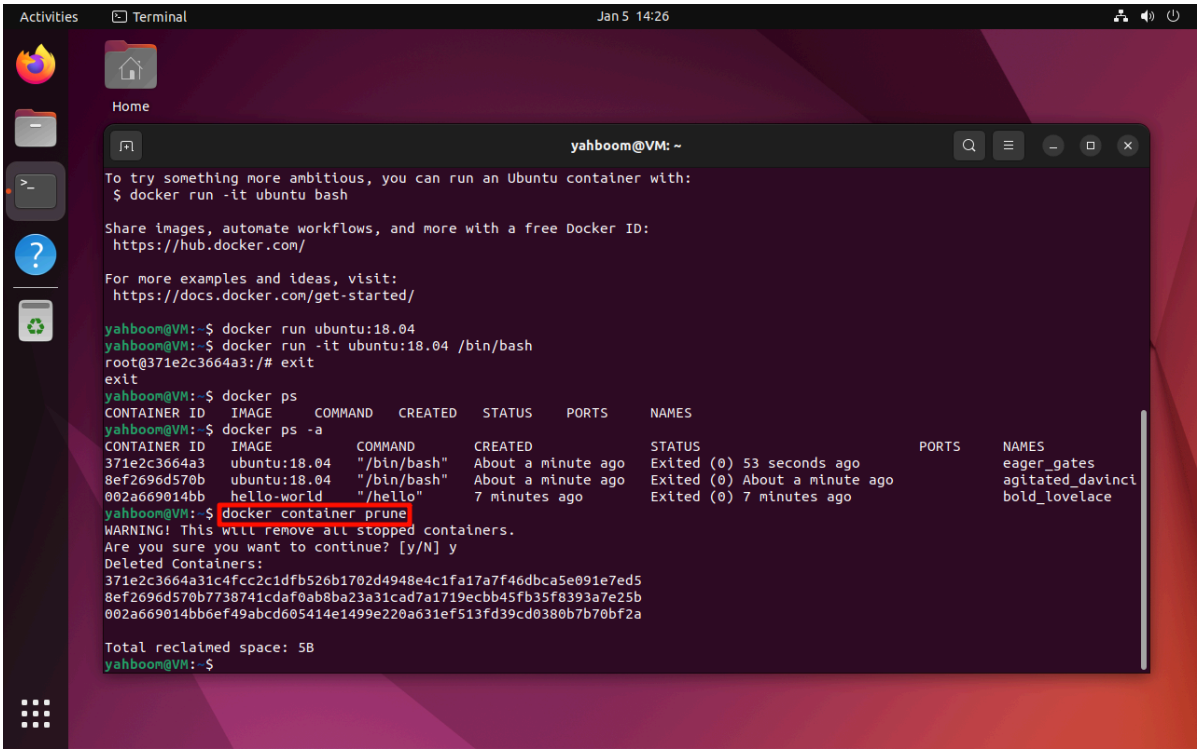


5. Clean Up Stopped Containers

This command deletes all stopped containers on the local machine and does not delete images. Before executing it, confirm that these containers contain no data that needs to be retained; Docker asks for confirmation again during execution.

> **docker container prune**

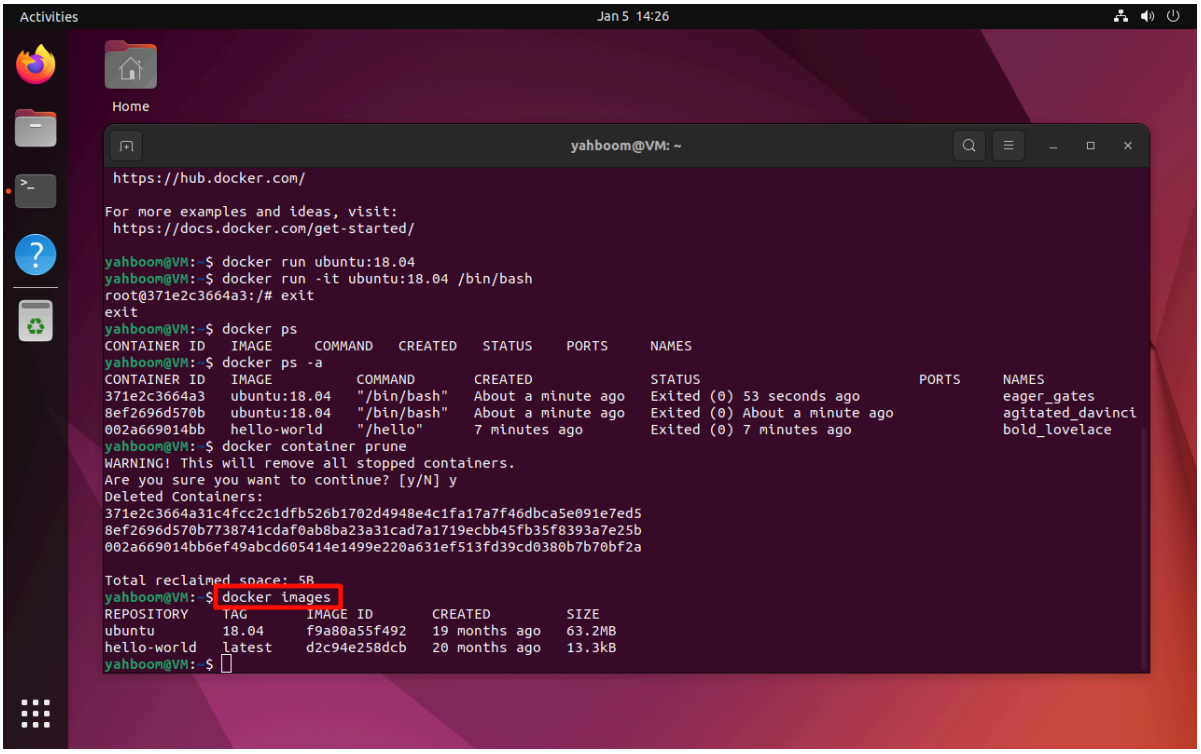
```
docker container prune
```



6. View Local Images

> **docker images**

```
docker images
```



7. Delete Images

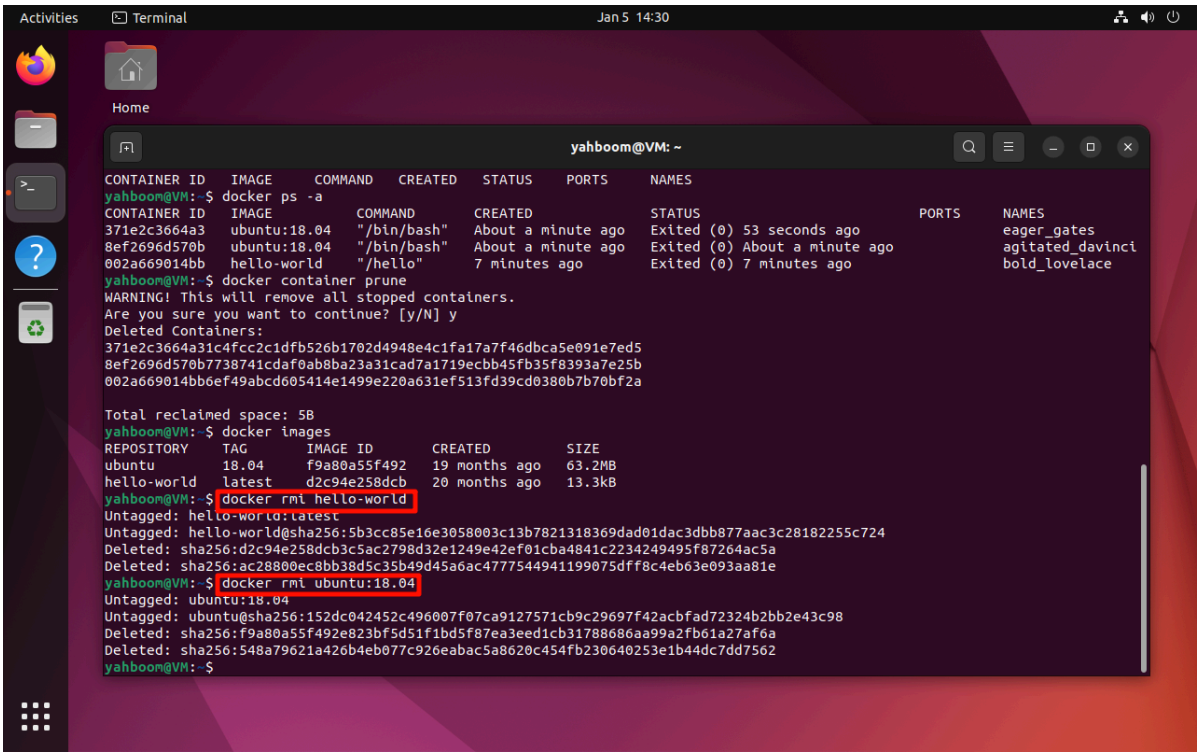
Note: If a container is still using this image, stop and delete that container first; only then can the image be removed.

> **docker rmi <image_name>**

```
docker rmi hello-world
```

> **docker rmi <image_name>:**

```
docker rmi ubuntu:18.04
```



8. Commit the Image

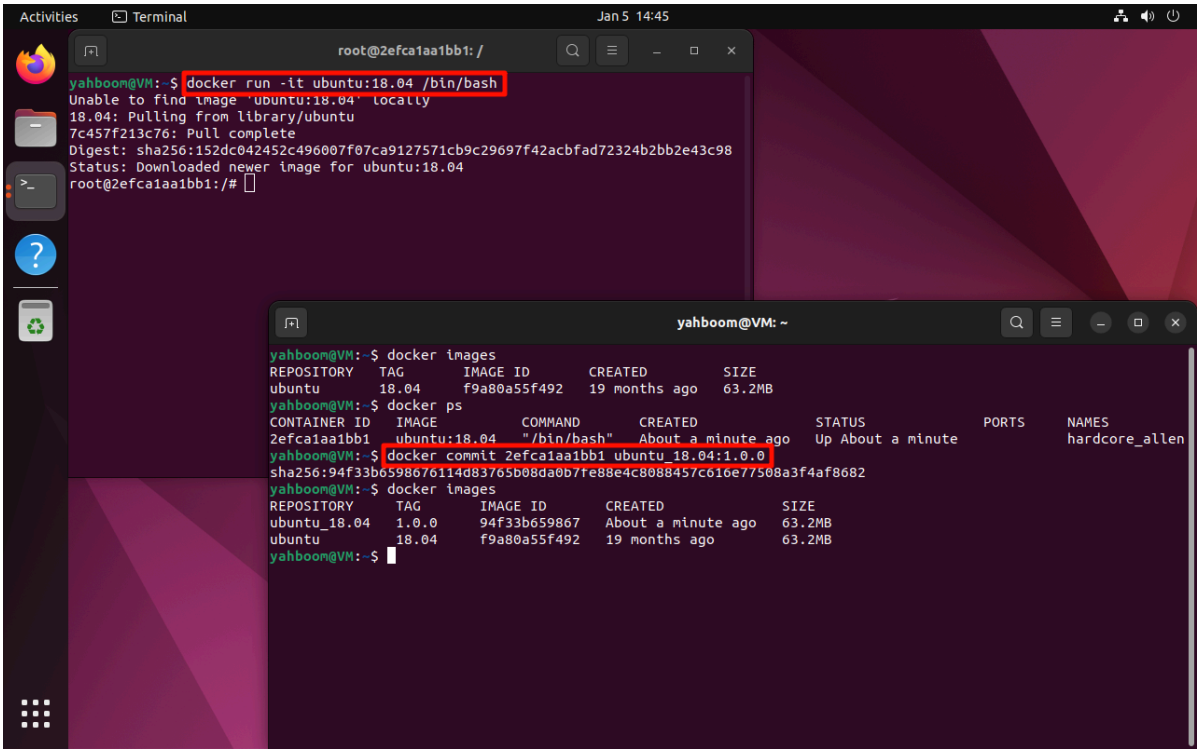
Pull and run the ubuntu:18.04 image in interactive mode

```
docker run -it ubuntu:18.04 /bin/bash
```

> **docker commit <image_name>:**

Note: Modify according to the actual `CONTAINER ID`

```
docker commit 2efca1aa1bb1 ubuntu_18.04:1.0.0
```



9. Stop the Container

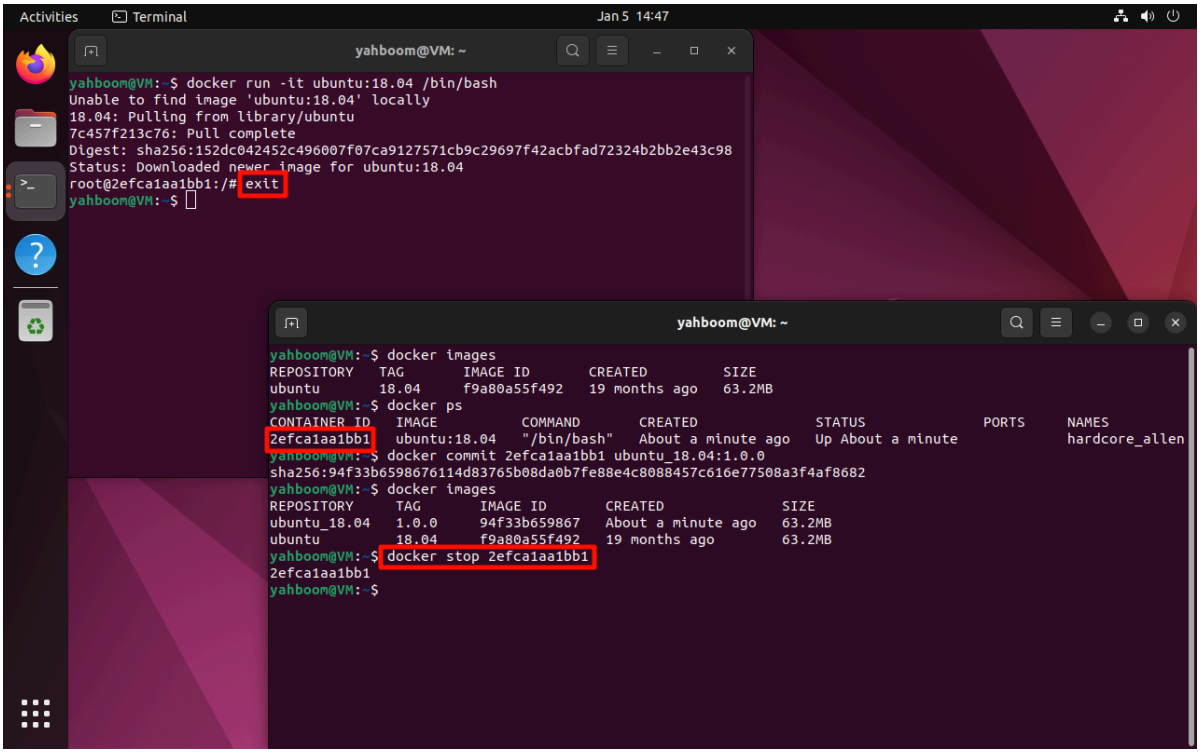
If the container is run in interactive mode and the terminal has entered the container, you can enter `exit` inside the container to stop it;

If the container is closed from the outside, you can use the `docker stop` command.

> **docker stop**

Note: Modify according to the actual `CONTAINER ID`

```
docker stop 2efca1aa1bb1
```



10. Multiple Terminals Entering the Same Container

Containers are isolated from each other. Each execution of `docker run` creates and starts a new container; to perform operations in the same container, use `docker exec` to enter that container.

Start a container based on the `ubuntu:18.04` image in interactive mode:

```
docker run -it ubuntu:18.04 /bin/bash
```

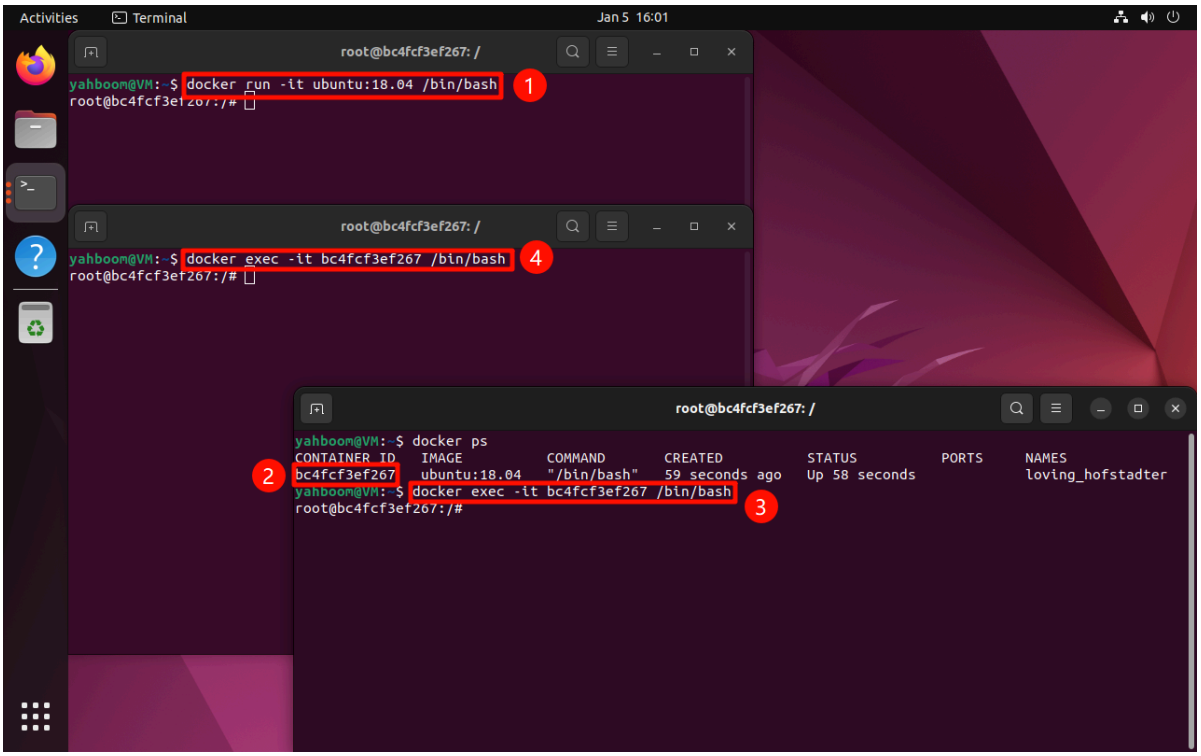
View the running container:

```
docker ps
```

> **docker exec**

Enter the running container in interactive mode: modify according to the actual `CONTAINER ID`

```
docker exec -it bc4fcf3ef267 /bin/bash
```


Activities
Terminal
Jan 5 16:01
root@bc4fcf3ef267: /
yahboom@VM:~$ docker run -it ubuntu:18.04 /bin/bash
root@bc4fcf3ef267:/#
1
root@bc4fcf3ef267: /
yahboom@VM:~$ docker exec -it bc4fcf3ef267 /bin/bash
root@bc4fcf3ef267:/#
4
root@bc4fcf3ef267: /
yahboom@VM:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
bc4fcf3ef267 ubuntu:18.04 "/bin/bash" 59 seconds ago Up 58 seconds loving_hofstadter
yahboom@VM:~$ docker exec -it bc4fcf3ef267 /bin/bash
root@bc4fcf3ef267:/#
2
3